
Protocol: NDIC HTTP push all deltas on occurrence

Release 1.0.1

Národní dopravní informační centrum (NDIC)

Aug 20, 2017

CONTENTS:

1	Quick Summary	2
2	Concepts	3
3	Platform independent messaging pattern	4
4	Implementation using HTTP POST	6
4.1	Roles and scenarios	6
4.2	Other features of HTTP	6
5	Testing delivery url from command line	8
5.1	Using curl command line tool	8
5.2	Using httpie command line tool	8

uri cz-ndic_push-v1.0

protocol 1.0

specification 1.0.1

Author Jan Vlčinský

Date 2016-05-24

Contact <http://registr.dopravniinfo.cz>

Organization NDIC

This document describes protocol, used to send data from provider to consumer at the moment, new version of data occurs. The protocol uses HTTP POST.

This protocol is used by NDIC to send updates of data in DDR format, however, it can be used by any other subject to send any kind of data in any format.

In this document we abbreviate the protocol name to PUSH.

QUICK SUMMARY

Data are to be sent from a provider to a consumer.

1. Consumer shall run HTTP server and give provider an url (called delivery url) to which the data are to be sent using HTTP POST (the message is the only part of request body).
2. Provider configures new subscription for given delivery url.
3. Provider initially sends all messages which were created in the past, but are still valid.
4. At the moment, changes relevant for given consumer occur, provider sends the changes to the consumer.

Provider shall send messages in order and in one thread.

Consumer shall return HTTP status 200.

In case there are problems to deliver a message (delivery url is not reachable or returned HTTP status code was other than 200), provider shall keep retrying delivering the message until success.

CONCEPTS

provider the party providing data to a consumer.

consumer the party consuming data provided by a provider.

delivery url a url provided by HTTP server at consumer supposed to accept HTTP POST requests with message being the only content of request body.

event information having identity and validity in time describing something. An event may be updated in time.

situation collection of events.

snapshot situation describing all relevant events for one moment in time.

message data being sent by provider to consumer describing some information.

delta message a message, describing changes of a situation. It includes details about creation, modification and deletion of one or more events.

subscription the fact, that some consumer is configured at provider to consume data on given delivery url.

session sequence of all messages provided during single subscription.

Provider provides delta messages. Initially, after subscription is created, all events which were created sooner but are still valid are sent via initial set of delta messages.

These delta messages allow consumer to build his own initial snapshot.

Later, as changes occur, further delta messages are sent and consumer shall be able updating the snapshot change by change.

PLATFORM INDEPENDENT MESSAGING PATTERN

Messages flow in one direction from provider to consumer.

Messages are of type “delta message”. It is assumed, complete sequence of messages in one session shall allow creating snapshot for given moment.

Provider takes care to send messages only in one thread to deliver them in order.

So called session starts at by a subscription (not arranged by this protocol) and provider sends one or more messages allowing to build snapshot for given moment. Consumer receives the messages and uses them to build own snapshot.

After initial set of messages, following messages are sent by the provider at the moment new update is created. Technically there is no difference between sending initial set of messages and delta messages sent on occurrence of a change.

Consumer shall save each received message and confirm, it was safely received.

If consumer has temporary problems to receive the message properly, it may reject to confirm successful message reception. However, it will serve only as request to resend the same message again. Consumer shall not assume, the message will get somehow improved or corrected by next attempt.

Rejecting a message blocks sending any further updates (following delta messages) until the rejected message is considered successfully delivered or it is considered expired.

If provider does not receive confirmation of message reception, it shall keep retrying until success. In extreme case it may assume there is no chance to succeed in future and stop sending any further messages. The protocol does not provide any mean to inform consumer about provider quitting further delivery of messages.

Consumer shall accept scenario, where some message is delivered multiple times.

If consumer miss some delta message, the only safe method to build valid snapshot is to start new session.

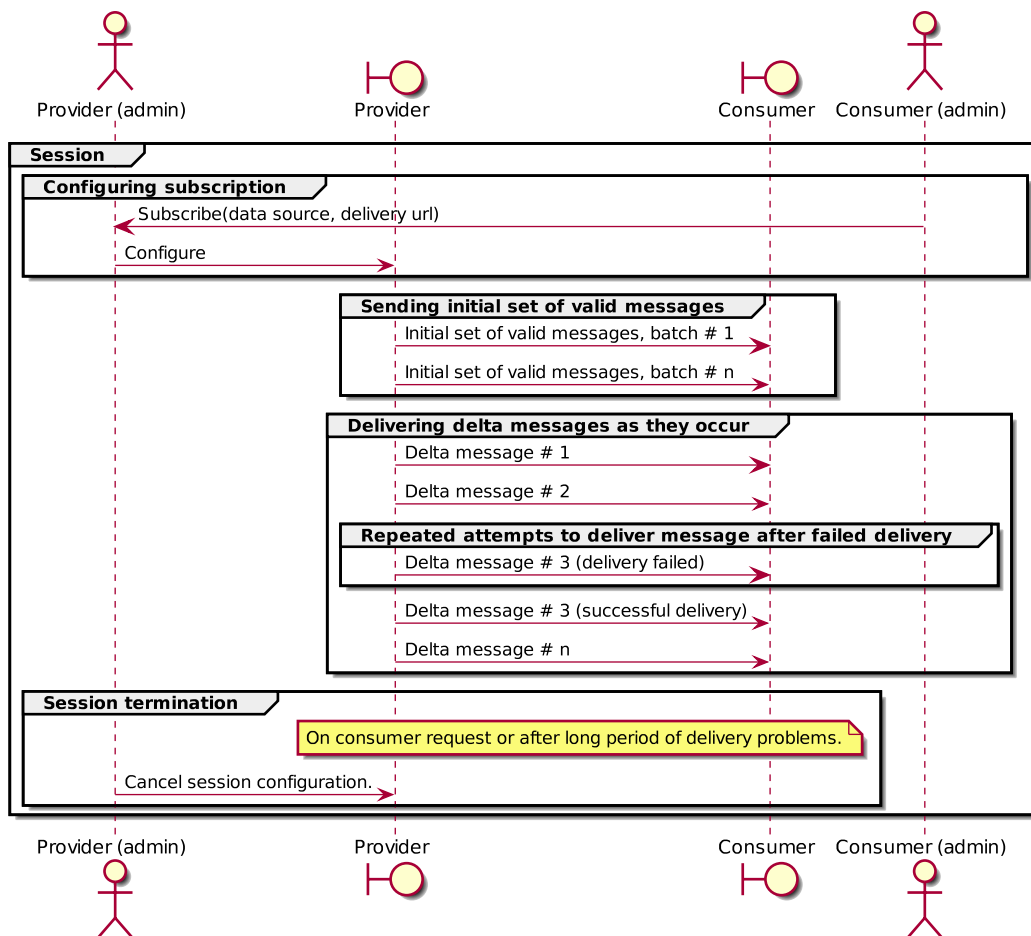


Fig. 3.1: Platform independent sequence diagram

IMPLEMENTATION USING HTTP POST

4.1 Roles and scenarios

Consumer must run web server and provide provider with url to post data to (delivery url).

Provider sends a message using HTTP POST as the only content of the body.

Warning: Do not use multipart-form data or urlencoded form data encoding!!!

Consumer confirms message reception by reporting HTTP status code 200. Any other status code is considered a request to resend the message.

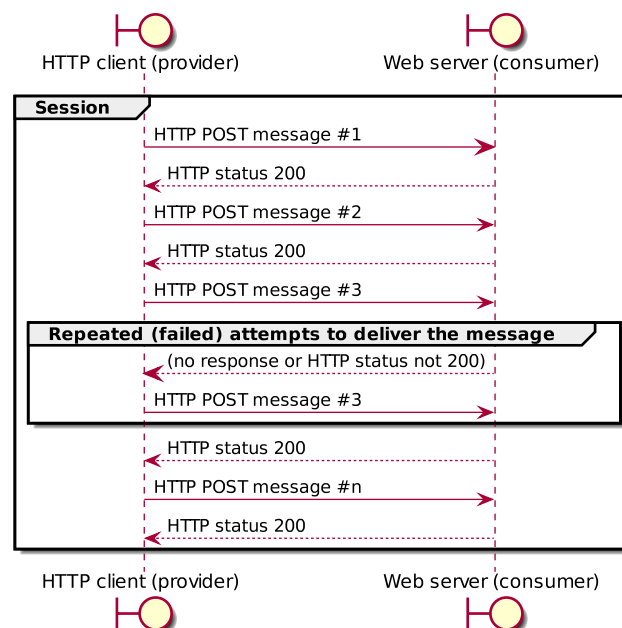


Fig. 4.1: Sequence diagram for implementation using HTTP

4.2 Other features of HTTP

authorization There is no authorization built in. Upon agreement, consumer may reject messages from other than agreed IP addresses.

compression Consumer may not force provider to send data in compressed form (using content-encoding: gzip). All content is delivered in plain without any content encoding.

content type Expected content type depends on format specification and agreement, it is not in scope of this protocol to require specific content type. In most cases the content type will be application/xml.

max size Default maximum size of posted content is 200 kB. Different content size may be agreed.

https HTTP server at consumer may use https protocol.

TESTING DELIVERY URL FROM COMMAND LINE

Following examples assume, your delivery url is <http://localhost:8080/upload/data> .

Create a file with name `data.xml`, content can be e.g.:

```
<root\>
```

5.1 Using curl command line tool

curl is popular command line tool for HTTP communication. For more see <https://en.wikipedia.org/wiki/CURL#curl>

Using curl, the data can be sent to the delivery url by:

```
$ curl -H "Content-Type: application/xml" -X POST -d @data.xml http://  
↪localhost:8080/upload/data
```

It sends HTTP POST request to <http://localhost:8080/upload/data> using (optional) header “*Content-Type: application/xml*” and using content of file *data.xml* as body of the *POST* request.

5.2 Using httpie command line tool

Another popular command line tool for sending HTTP requests is httpie providing executable named http. For more see: <https://github.com/jkbrzt/httpie>

Using http executable, the data can be sent to the delivery url by:

```
$ http POST http://localhost:8080/upload/data <data.xml
```